

Visione Artificiale

Assignment 3

Salvo Portuesi | salvo.portuesi@community.unipa.it

Architettura del sistema

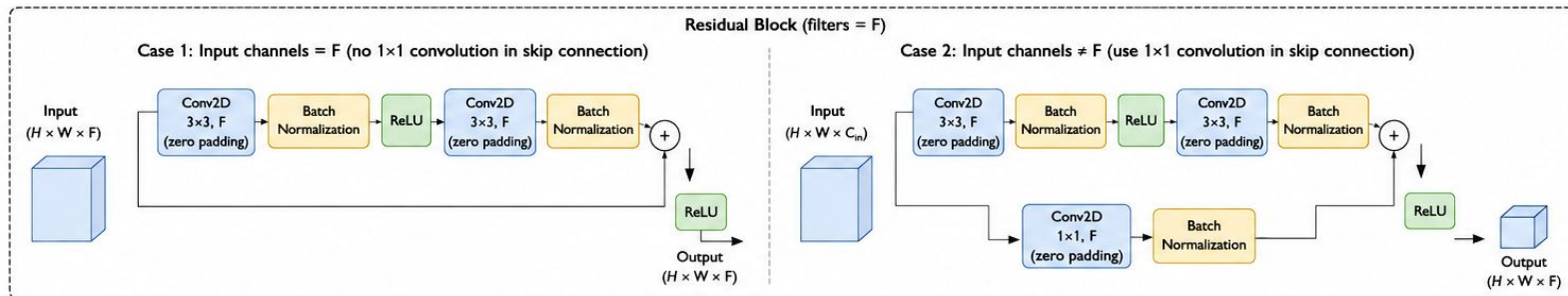
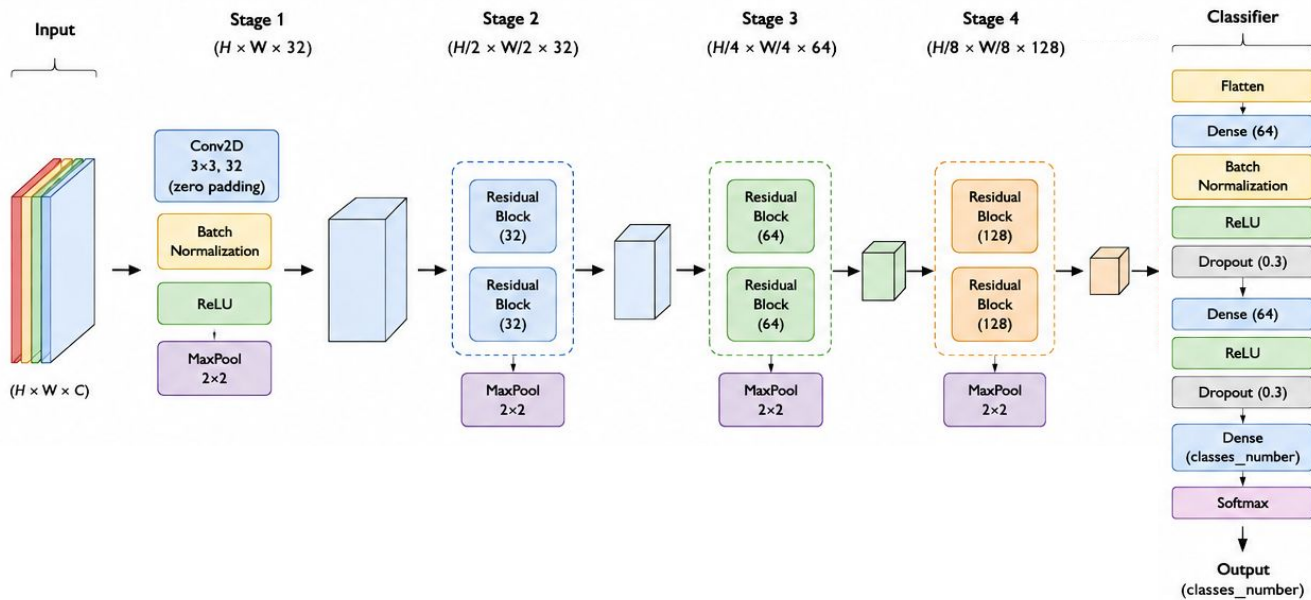
Il sistema è composto da quattro moduli principali, i quali si dividono le responsabilità nel modo seguente:

- A. Definizione dell'architettura della rete neurale e del blocco residuale, preprocessing delle immagini e classificazione di un'immagine fissati i pesi di un modello;
- B. Gestione del pre-training della rete convoluzionale e costruzione dei dataset Keras di training/validation a partire dal dataset AID;
- C. Gestione del fine-tuning e del training da zero della rete convoluzionale, costruzione dei dataset Keras di training/validation/testing a partire dal dataset UC Merced Land Use, confronto tra le metriche di performance dei modelli e scelta della configurazione migliore;
- D. Data augmentation, lettura ottimizzata dei dataset con generatori Python e funzioni di utilità.

Gli ultimi tre moduli menzionati sono necessari solo durante lo sviluppo e l'addestramento.

Modulo A: inferenza e rete convoluzionale residuale

- In fase di inferenza vengono caricati i pesi di un modello già addestrato e viene restituita la classe con probabilità stimata più elevata.
- Le immagini vengono pre-processate applicando il ridimensionamento a 100x100 pixel e la normalizzazione ImageNet.
- La rete riprende i principi fondamentali di ResNet (skip connections, batch normalization, ReLU e aumento progressivo dei filtri) ma li declina in una versione leggera da circa 1M di parametri contro i 25M di ResNet-50.
- Vengono usati esattamente 999,445 parametri totali, di cui 300,501 appartengono al MLP. In pre-training i parametri totali sono invece 1,000,030 a causa del numero maggiore di classi presenti nel dataset AID. Pertanto, le due reti differiscono architetturalmente solo per l'ultimo layer.
- Il diagramma presente nella slide successiva mostra l'architettura della rete convoluzionale residuale usata.

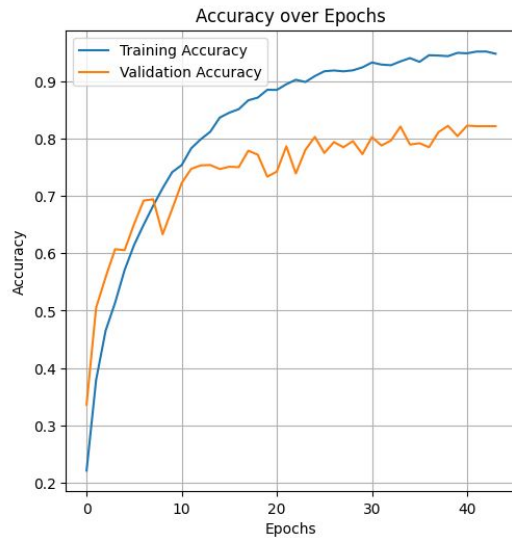
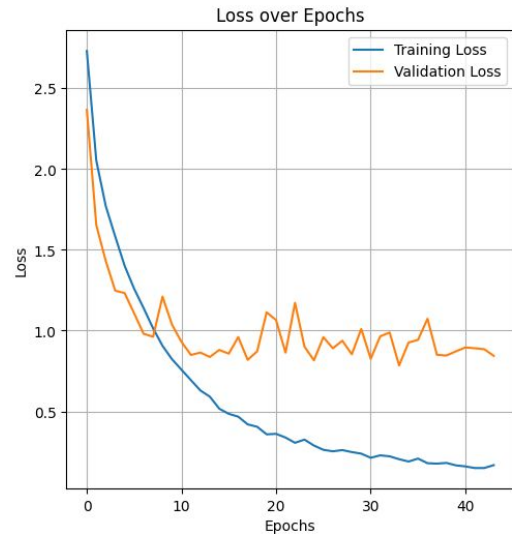


Modulo A: inferenza e rete convoluzionale residuale

- Tutti i layer convoluzionali utilizzano lo zero padding per preservare le dimensioni spaziali.
- Nei blocchi residuali della CNN, se il numero di filtri del ramo principale differisce dai canali in ingresso, la skip connection utilizza una convoluzione 1x1 per allineare le dimensioni.
- Il numero di filtri usati nei blocchi residuali raddoppia in ogni stadio compensando la riduzione di risoluzione spaziale introdotta dal max pooling.
- Il batch normalization permette di accelerare la convergenza e stabilizzare l'addestramento, permettendo di usare learning rate più elevati.
- L'uso del dropout nel MLP permette di limitare l'overfitting senza penalizzare la capacità e impedisce ai neuroni di sviluppare una co-adattamento eccessivo.

Modulo B: pre-training

- Il dataset viene suddiviso in training set (70%) e validation set (30%) con campionamento stratificato. Si costruiscono quindi i relativi dataset Keras, i quali suddividono i campioni in batch.
- Si utilizzano: Adam con learning rate pari a $1e-3$ come ottimizzatore, categorical cross-entropy come funzione di loss, accuratezza come metrica di riferimento e 200 epoche di addestramento.
- Per ogni immagine del training set ne viene generata un'altra tramite data augmentation.
- Viene utilizzato l'early stopping, il quale interrompe l'addestramento se la loss sul validation set non migliora per più di 10 epoche. Come mostra la figura in alto, questa tecnica ha limitato l'overfitting arrestando il training dopo 44 epoche.
- Un callback di Keras salva i pesi del modello corrispondenti alla minima loss sul validation set e non necessariamente quelli dell'ultima epoca.



Modulo C: fine-tuning e del training da zero

- Il dataset viene suddiviso in training set (70%), validation set (15%) e testing set (15%) con campionamento stratificato.
- Poiché il numero di classi del dataset di training è diverso da quello di pre-training, bisogna prima caricare i pesi del modello del pre-training e poi sostituire l'ultimo layer fully connected del MLP.
- Il fine-tuning è stato eseguito in tre configurazioni: head fine-tuning (solo layer fully connected addestrabili: 300,629 parametri), partial fine-tuning (layer a partire dal quarto stadio addestrabili: 826,837) e full fine-tuning (tutti i layer addestrabili: 997,077 parametri).
- Viene eseguito il training da zero usando la stessa architettura precedente (997,077 parametri addestrabili).
- Per ogni immagine del training set ne viene generata un'altra tramite data augmentation, aumentando la variabilità dei dati disponibili e favorendo una convergenza più rapida. Si è visto che fissare un numero più alto di immagini generate con data augmentation peggiora le performance sul validation set.
- Per il fine-tuning si utilizza un learning rate pari a $1e-4$, mentre per il training da zero uno pari a $1e-3$. Gli altri iper-parametri sono uguali a quelli usati in pre-training.

Modulo D: data augmentation e lettura ottimizzata dei dataset

- Per aumentare la capacità di generalizzazione, durante il training viene sempre applicata la data augmentation mediante rotazioni casuali con ampiezza massima di $\pm 10^\circ$ e capovolgimenti orizzontali o verticali.
- Per garantire una partizione fissa tra training, validation e testing senza caricare simultaneamente tutte le immagini in memoria, viene creata una lista contenente i percorsi delle immagini e le relative etichette.
- Una classe dedicata fornisce tramite un generatore Python i campioni contenenti le singole immagini e le label associate. In questa fase vengono applicati pre-processing, data augmentation e codifica one-hot delle etichette.
- La classe menzionata sopra viene usata nei moduli B e C per costruire i dataset Keras, i quali permettono la suddivisione in batch da fornire alla rete neurale.

Risultati quantitativi e modello selezionato

Sul validation set sono state ottenute le seguenti prestazioni:

	Head Fine-tuned	Partial Fine-tuned	Full Fine-tuned	Trained from Zero
F1-Score (macro)	0.8142	0.8868	0.9174	0.8832

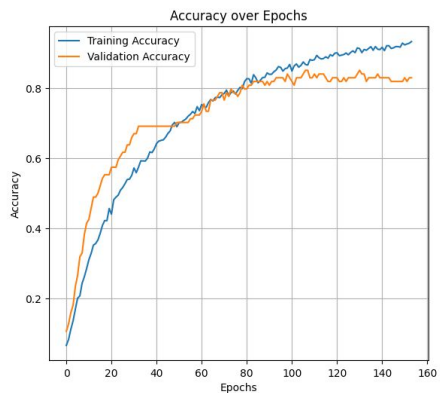
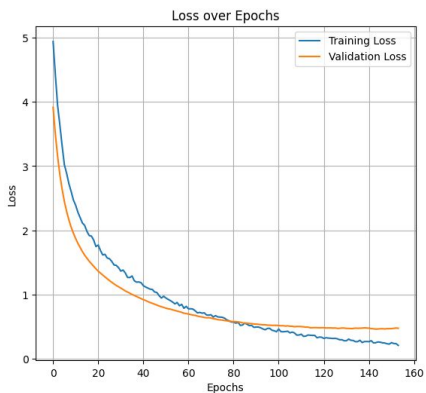
Il fine-tuning del solo MLP si dimostra troppo limitante. In particolare, il backbone rimane ancorato alla distribuzione del dataset AID e lo spazio delle feature in ingresso al classificatore non viene ottimizzato.

Il fine-tuning parziale raggiunge prestazioni molto simili a quelle del modello allenato da zero, suggerendo che i layer congelati (stadi 1, 2 e 3) estraggono feature di basso livello già sufficientemente generali, mentre i layer profondi codificano feature semantiche specifiche del dominio.

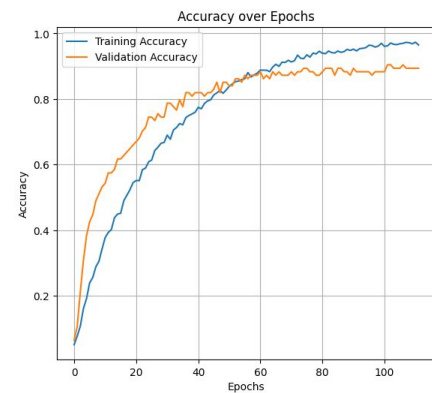
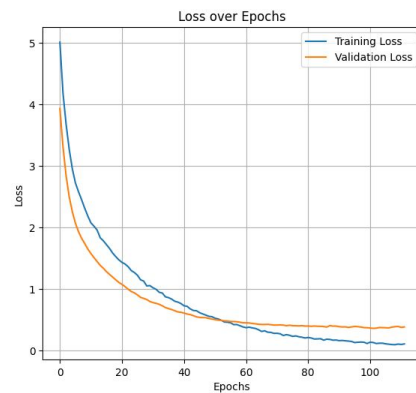
La strategia più efficace risulta essere il fine-tuning completo, nella quale il training parte da pesi già significativi anziché da rumore casuale, evitando le forti fluttuazioni osservate nel modello addestrato da zero.

Nella slide successiva si mostrano gli andamenti di loss e accuracy durante l'addestramento per il training set e per il validation set.

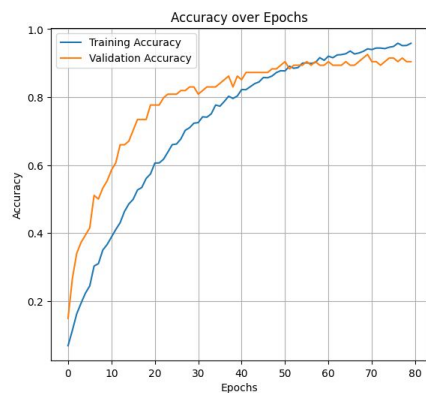
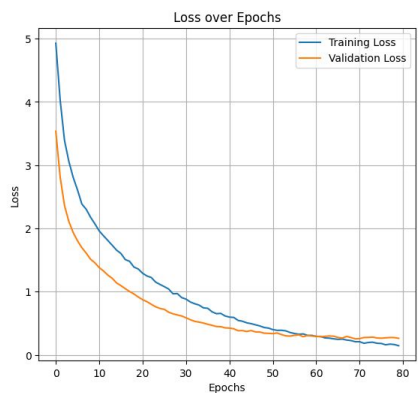
Head Fine-tuned



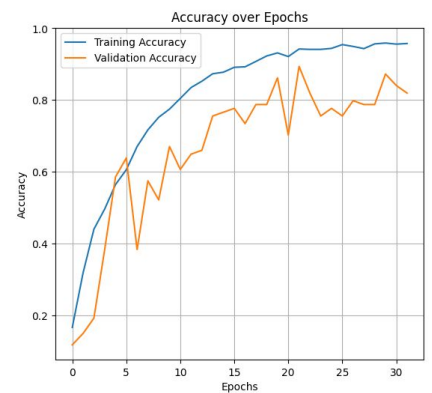
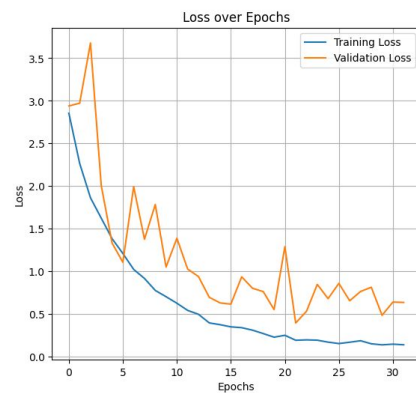
Partial Fine-tuned



Full Fine-tuned



Trained from Zero



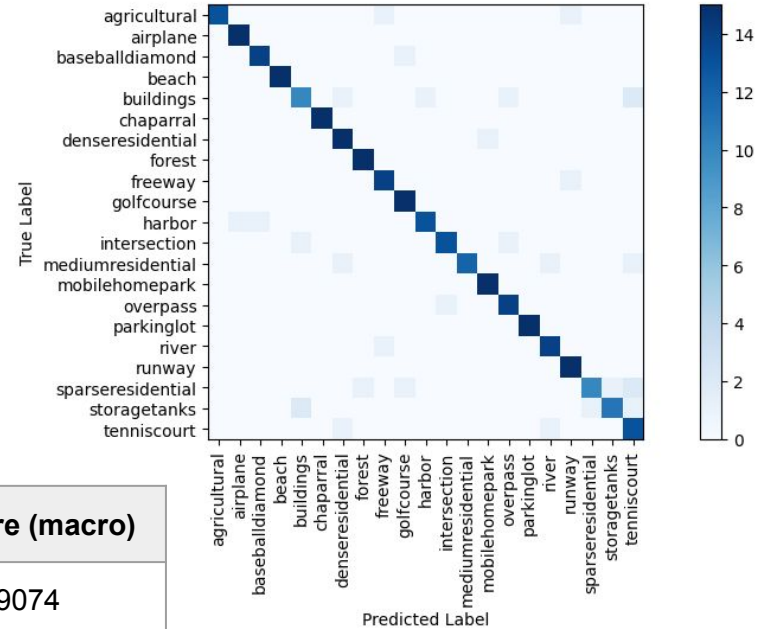
Risultati quantitativi e modello selezionato

- In tutti i quattro casi l'early stopping interrompe l'addestramento prima dell'esecuzione delle 200 epoche. In particolare, si osserva che nei modelli fine-tuned il numero di epoche utilizzate aumenta al diminuire dei parametri addestrabili. Ciò accade a causa della minore capacità di assorbimento delle informazioni.
- In tutti i casi si osserva che, mentre la validation loss si stabilizza, la training loss continua a decrescere: tale comportamento è un chiaro sintomo di overfitting.
- La loss del modello addestrato da zero mostra marcate fluttuazioni sul validation set, evidenziando una minore stabilità in fase di apprendimento e confermando il contributo del pre-training adottato negli altri modelli. Tuttavia, l'addestramento necessita solo di 32 epoche grazie anche al training rate più elevato.

Risultati quantitativi e modello selezionato

Di fianco si trova la matrice di confusione relativa al modello migliore (full fine-tuned) sul test set, mentre in basso sono presenti le relative metriche rilevate.

L'F1-Score mostra un miglioramento del 18% rispetto alla support vector machine utilizzata nell'Assignment 2.



Accuracy	Precision (macro)	Recall (macro)	F1-Score (macro)
0.9051	0.9098	0.9050	0.9074

Risultati qualitativi

È interessante notare che gli errori di classificazione sono più comuni nel caso di classi semanticamente simili. Ad esempio, la misclassificazione tra *mediumresidential* e *denserresidential* è una delle più comuni. Ne è un esempio l'immagine del test set in alto a sinistra, la cui classe reale è *mediumresidential*.

Casi come quello della figura in alto a destra risultano essere intrinsecamente complessi perché composti da più elementi differenti. In questo caso, la classe reale dell'immagine è *tenniscourt*, ma l'etichetta predetta è *denserresidential* con una probabilità del 57%. Seguono le classi *mediumresidential* (12%), *intersection* (7%), *buildings* (6%) e *tenniscourt* (5%).

Invece, l'immagine in basso a sinistra viene classificata correttamente come *tenniscourt* con una probabilità pari all'80% nonostante siano presenti molti elementi di disturbo.

Analogamente, l'immagine in basso a destra viene classificata correttamente come *intersection* con una probabilità pari al 99%, tuttavia quest'ultima faceva parte del training set. La presenza del campo da tennis fa sorgere dei dubbi sul sovradattamento.



Possibili miglioramenti del sistema

- Poiché il dataset UC Merced Land Use contiene solamente 2100 immagini, lo split fisso tra training, validation e testing espone al rischio di valutazioni influenzate dal caso. Invece, la K-Fold Cross Validation potrebbe fornire stime di performance più robuste.
- Le classi presenti nel dataset sono bilanciate, ma in un utilizzo reale del sistema alcune categorie potrebbero essere più comuni di altre. Un miglioramento possibile che non influisce sul training sarebbe quello di moltiplicare le probabilità ottenute dalla rete neurale per dei prior.
- Gli iper-parametri che determinano l'architettura della rete sono stati scelti empiricamente, ma si potrebbe usare la libreria KerasTuner per ottimizzarli automaticamente.
- La classificazione multi-classe forza il modello a scegliere un'unica etichetta, ignorando il fatto che un'immagine aerea contiene spesso più elementi. Questo crea ambiguità sia per il modello in fase di addestramento, sia nella valutazione dei casi d'errore. Ne è un esempio l'immagine riportata in basso a destra nella slide precedente. Una possibile soluzione consiste nell'utilizzo di un dataset in cui a ogni immagine sono associate più classi.

Sitografia e tool utilizzati

- Documentazione TensorFlow: [Dataset.from_generator](#).
- Documentazione Keras: [MaxPooling2D](#), [ModelCheckpoint](#), eccetera.
- Gemini: [Keras Convolutional Layer Order](#).
- ChatGPT: [Neural Network Diagram](#) per una figura opportunamente corretta presente in una slide.
- GPT-5.2-Codex: per l'implementazione iniziale delle funzioni `augment_image()`, per la scrittura del file [README.md](#) e per l'implementazione delle funzioni `parse_args()` e `show_training_history_over_epochs()`.
- Autocompletamento del codice in Visual Studio Code.
- Risorse varie: [Keras Callbacks and How to Save Your Model from Overtraining](#), [Understanding Dropout in Deep Learning: A Guide to Reducing Overfitting](#)